Conversation with Gemini

Take a tutor role and explain in detail the following python code to a software engineer who is not into AI:

```
# run_all_base_models.py

# Robust driver to train and evaluate multiple classifiers on all cleaned
CIC-IDS-2018 datasets
# across defined feature groups. Computes overall and per-attack metrics and
saves results.

import pandas as pd
from pathlib import Path
from joblib import Parallel, delayed
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, ExtraTreesClassifier
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from catboost import CatBoostClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import LinearSVC
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, f1_score, precision_score,
recall_score, confusion_matrix
import json
import time

# --- Configuration ---
CLEAN_DIR = Path("cleaned")
RESULTS_DIR = Path("results") / "base_models"
RESULTS_DIR.mkdir(parents=True, exist_ok=True)
OVERALL_CSV = RESULTS_DIR / "overall_metrics.csv"
PER_ATTACK_CSV = RESULTS_DIR / "per_attack_metrics.csv"

# Classifiers and parameter grids
MODELS = {
    "LogisticRegression": (LogisticRegression(solver='saga', max_iter=1000,
random_state=42), {"C": [0.01, 0.1, 1]}),
    "DecisionTree": (DecisionTreeClassifier(random_state=42), {"max_depth":
[5, 10, 20]}),
    "RandomForest": (RandomForestClassifier(random_state=42, n_jobs=1),
{"n_estimators": [100, 200], "max_depth": [None, 10, 20]}),
    "ExtraTrees": (ExtraTreesClassifier(random_state=42, n_jobs=1),
{"n_estimators": [100, 200], "max_depth": [None, 10, 20]}),
    "XGBoost": (XGBClassifier(use_label_encoder=False, eval_metric='logloss',
random_state=42, n_jobs=1), {"learning_rate": [0.1, 0.01], "n_estimators":
[100, 200], "max_depth": [3, 5]}),
    "LightGBM": (LGBMClassifier(random_state=42, n_jobs=1), {"learning_rate":
[0.1, 0.01], "n_estimators": [100, 200], "num_leaves": [31, 50]}),
    "CatBoost": (CatBoostClassifier(verbose=0, random_state=42), {"depth":
[4, 6], "iterations": [100, 200], "learning_rate": [0.1, 0.01]}),
```

```
    "KNN": (KNeighborsClassifier(), {"n_neighbors": [3, 5, 7]}),
    "LinearSVC": (LinearSVC(max_iter=10000, random_state=42), {"C": [0.1, 1,
10]}),
    "MLP": (MLPClassifier(max_iter=500, random_state=42),
{"hidden_layer_sizes": [(50,), (100,)], "alpha": [1e-4, 1e-3]})
}

# Feature categories (must match preprocessor)
FEATURE_CATEGORIES = {
    "flow_metrics": ["Flow Duration", "Flow Byts/s", "Flow Pkts/s", "Flow IAT
Mean", "Flow IAT Std", "Flow IAT Max", "Flow IAT Min"],
    "packet_size_stats": ["Tot Fwd Pkts", "Tot Bwd Pkts", "TotLen Fwd Pkts",
"TotLen Bwd Pkts", "Fwd Pkt Len Max", "Fwd Pkt Len Min", "Fwd Pkt Len Mean",
"Fwd Pkt Len Std", "Bwd Pkt Len Max", "Bwd Pkt Len Min", "Bwd Pkt Len Mean",
"Bwd Pkt Len Std", "Pkt Len Min", "Pkt Len Max", "Pkt Len Mean", "Pkt Len
Std", "Pkt Len Var", "Pkt Size Avg", "Fwd Seg Size Avg", "Bwd Seg Size Avg"],
    "timing_iat": ["Fwd IAT Tot", "Fwd IAT Mean", "Fwd IAT Std", "Fwd IAT
Max", "Fwd IAT Min", "Bwd IAT Tot", "Bwd IAT Mean", "Bwd IAT Std", "Bwd IAT
Max", "Bwd IAT Min"],
    "flags_and_protocol": ["FIN Flag Cnt", "SYN Flag Cnt", "RST Flag Cnt",
"PSH Flag Cnt", "ACK Flag Cnt", "URG Flag Cnt", "CWE Flag Count", "ECE Flag
Cnt", "Fwd PSH Flags", "Bwd PSH Flags", "Fwd URG Flags", "Bwd URG Flags",
"Protocol", "Dst Port", "Init Fwd Win Byts", "Init Bwd Win Byts"],
    "rates_and_ratios": ["Down/Up Ratio", "Fwd Pkts/s", "Bwd Pkts/s", "Fwd
Byts/b Avg", "Fwd Pkts/b Avg", "Fwd Blk Rate Avg", "Bwd Byts/b Avg", "Bwd
Pkts/b Avg", "Bwd Blk Rate Avg"],
    "connection_activity": ["Subflow Fwd Pkts", "Subflow Fwd Byts", "Subflow
Bwd Pkts", "Subflow Bwd Byts", "Fwd Act Data Pkts", "Active Mean", "Active
Std", "Active Max", "Active Min", "Idle Mean", "Idle Std", "Idle Max", "Idle
Min"]
}

# Function to train and evaluate one combination
def run_experiment(params):
    file_path, group_name, feats, model_name, (estimator, grid) = params
    start_time = time.time()
    print(f"\n--- Starting {model_name} on {file_path.name}/{group_name} at
{time.ctime()} ---")

    # Load data
    df = pd.read_csv(file_path, low_memory=False)
    X = df[feats]
    y = df['Label']

    # Split
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, stratify=y, test_size=0.3, random_state=42
    )

    # Grid search
    print(f"Grid-searching {model_name} (params: {list(grid.keys())})...")
    gs = GridSearchCV(estimator, grid, cv=3, scoring='f1', n_jobs=1)
    gs.fit(X_train, y_train)
    best = gs.best_estimator_
    print(f"Best params for {model_name}: {gs.best_params_}")

    # Predict
```

```
    print(f"Evaluating {model_name}...")
    y_pred = best.predict(X_test)

    # Overall metrics
    overall = {
        'file': file_path.name,
        'attack_type': file_path.stem,
        'feature_group': group_name,
        'model': model_name,
        'best_params': json.dumps(gs.best_params_),
        'accuracy': accuracy_score(y_test, y_pred),
        'f1_score': f1_score(y_test, y_pred),
        'precision': precision_score(y_test, y_pred),
        'recall': recall_score(y_test, y_pred)
    }

    # Per-attack metrics
    per_attack = []
    df_test = df.loc[y_test.index].copy()
    df_test['_pred'] = y_pred
    for atk in sorted(df_test['Attack Type'].unique()):
        mask = df_test['Attack Type'] == atk
        if mask.sum() < 2:
            continue
        pa_y = df_test.loc[mask, 'Label']
        pa_pred = df_test.loc[mask, '_pred']
        per_attack.append({
            'file': file_path.name,
            'attack_type': atk,
            'feature_group': group_name,
            'model': model_name,
            'accuracy': accuracy_score(pa_y, pa_pred),
            'f1_score': f1_score(pa_y, pa_pred),
            'precision': precision_score(pa_y, pa_pred),
            'recall': recall_score(pa_y, pa_pred)
        })

    elapsed = time.time() - start_time
    print(f"--- Completed {model_name} on {file_path.name}/{group_name} in
{elapsed:.1f}s ---")
    return overall, per_attack

# Build task list
tasks = []
for file_path in sorted(CLEAN_DIR.glob("*.csv")):
    print(f"Queueing file: {file_path.name}")
    for group, feats in FEATURE_CATEGORIES.items():
        sample_cols = pd.read_csv(file_path, nrows=1,
low_memory=False).columns
        avail = [f for f in feats if f in sample_cols]
        if not avail:
            print(f"  Skipping group {group}: no features present.")
            continue
        print(f"  Feature group: {group} ({len(avail)} features)")
        for model_name, model_def in MODELS.items():
            tasks.append((file_path, group, avail, model_name, model_def))
```

```
print(f"Total tasks to run: {len(tasks)}")

# Run in parallel
results = Parallel(n_jobs=3)(delayed(run_experiment)(t) for t in tasks)

# Separate results
overall_list, per_attack_list = zip(*results)

# Save DataFrames
print(f"Saving overall metrics to {OVERALL_CSV}")
pd.DataFrame(overall_list).to_csv(OVERALL_CSV, index=False)
print(f"Saving per-attack metrics to {PER_ATTACK_CSV}")
pd.DataFrame([row for sub in per_attack_list for row in
sub]).to_csv(PER_ATTACK_CSV, index=False)

print("All done!")
```

Hello! As your tutor, I'm here to walk you through this Python script. Even without an AI background, you, as a software engineer, will find many familiar concepts here, especially around data processing, modularity, and parallel execution.

This script, `run_all_base_models.py`, is designed to systematically *train and evaluate* various machine learning models on a set of network traffic data. The goal is to see how different models perform when trying to detect different types of network attacks, using different subsets of features from the data. Think of it as a comprehensive benchmarking suite for classification models in a cybersecurity context.

Let's break down the code section by section.

---

1. Shebang and Docstrings (Implicit)

While not explicitly in this snippet, typically a Python script would start with a "shebang" line like `#!/usr/bin/env python3` to indicate the interpreter. The comments at the top (`# run_all_base_models.py`, etc.) serve as a high-level docstring, explaining the script's purpose: it's a "robust driver" for training and evaluating multiple classifiers on "cleaned CIC-IDS-2018 datasets" (a common benchmark dataset for intrusion detection), using various "feature groups," and saving the performance metrics.

---

2. Imports: Your Toolbox for Data Science

Python
```
import pandas as pd
from pathlib import Path
from joblib import Parallel, delayed
from sklearn.model_selection import train_test_split, GridSearchCV
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, ExtraTreesClassifier
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from catboost import CatBoostClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import LinearSVC
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, f1_score, precision_score,
recall_score, confusion_matrix
import json
import time
```

This section brings in all the necessary libraries. Think of these as specialized tools you're grabbing from your software engineering utility belt:

- **`pandas as pd`**: This is the go-to library for data manipulation and analysis. It provides DataFrames, which are like super-powered spreadsheets in Python, perfect for handling tabular data. You'll see it used for loading CSV files and managing results.
- **`pathlib.Path`**: A modern way to handle file paths. It makes working with files and directories more object-oriented and less error-prone than string-based paths.
- **`joblib.Parallel, delayed`**: These are for *parallel processing*. `joblib` allows you to execute functions in parallel across multiple CPU cores, which is crucial for speeding up computationally intensive tasks like training many machine learning models.
- **`sklearn.model_selection.train_test_split, GridSearchCV`**: From `scikit-learn`, the most popular machine learning library in Python.
    - `train_test_split`: Divides your dataset into training and testing sets. This is fundamental to evaluate a model on unseen data.
    - `GridSearchCV`: A powerful tool for "hyperparameter tuning." It automates the process of trying out different configurations (hyperparameters) for a model to find the ones that yield the best performance.
- **`sklearn.linear_model.LogisticRegression`**, **`sklearn.tree.DecisionTreeClassifier`**, etc.: These are the actual machine learning model classes. Each represents a different algorithm:
    - **Logistic Regression**: A linear model used for binary classification.
    - **Decision Tree**: A tree-like model that makes decisions based on features.
    - **Random Forest**, **Extra Trees**: Ensemble methods that build multiple decision trees and combine their predictions, generally more robust than a single tree.
    - **XGBoost**, **LightGBM**, **CatBoost**: Highly optimized, gradient-boosting frameworks known for their performance in competitive machine learning. These are often called "AI" or "ML" models.
    - **K-Nearest Neighbors (KNN)**: A non-parametric model that classifies points based on the majority class of their nearest neighbors.
    - **LinearSVC (Support Vector Classifier)**: A linear model for classification, based on Support Vector Machines.
    - **MLPClassifier (Multi-layer Perceptron)**: A basic type of neural network.

- **`sklearn.metrics.accuracy_score`, `f1_score`, `precision_score`, `recall_score`, `confusion_matrix`**: Functions to calculate various performance metrics of your classification models.
    - `accuracy_score`: Proportion of correctly classified instances.
    - `f1_score`: Harmonic mean of precision and recall, good for imbalanced datasets.
    - `precision_score`: Proportion of positive identifications that were actually correct.
    - `recall_score`: Proportion of actual positives that were identified correctly.
    - `confusion_matrix`: A table summarizing classification performance. (Though it's imported, it's not explicitly used in the final metric calculations presented in the `run_experiment` function, but it's a common tool for deeper analysis).
- **`json`**: For working with JSON data, specifically for saving the "best parameters" of a model.
- **`time`**: For timing the execution of different parts of the script.

---

3. Configuration: Setting up Your Environment

Python
```
# --- Configuration ---
CLEAN_DIR = Path("cleaned")
RESULTS_DIR = Path("results") / "base_models"
RESULTS_DIR.mkdir(parents=True, exist_ok=True)
OVERALL_CSV = RESULTS_DIR / "overall_metrics.csv"
PER_ATTACK_CSV = RESULTS_DIR / "per_attack_metrics.csv"
```

This section defines constants and sets up the directory structure for input and output.

- **`CLEAN_DIR = Path("cleaned")`**: Specifies the directory where the cleaned input datasets are located. `Path("cleaned")` creates a `Path` object representing the "cleaned" folder.
- **`RESULTS_DIR = Path("results") / "base_models"`**: Defines the output directory for results. `Path("results") / "base_models"` is a clean way to join paths (e.g., `results/base_models`).
- **`RESULTS_DIR.mkdir(parents=True, exist_ok=True)`**: Creates the `results/base_models` directory if it doesn't exist.
    - `parents=True`: Creates any necessary parent directories.
    - `exist_ok=True`: Prevents an error if the directory already exists.
- **`OVERALL_CSV`, `PER_ATTACK_CSV`**: Define the filenames for the two output CSV files that will store the aggregated results.

---

4. Classifiers and Parameter Grids (`MODELS`)

Python

```
# Classifiers and parameter grids
MODELS = {
    "LogisticRegression": (LogisticRegression(solver='saga', max_iter=1000,
random_state=42), {"C": [0.01, 0.1, 1]}),
    # ... (other models) ...
}
```

This is a dictionary (`MODELS`) that holds the heart of your machine learning experiments. Each entry defines a specific classifier and the hyperparameters you want to try for it:

- **Key (e.g., `"LogisticRegression"`)**: The name of the model, used for reporting.
- **Value (a tuple)**:
  - **First element**: An *instance* of the classifier with some initial settings.
    - `random_state=42`: This is a common practice to ensure *reproducibility*. If you run the code again with the same `random_state`, you'll get the exact same results for algorithms that involve randomness (like splitting data or initializing weights).
    - `n_jobs=1`: For some models like Random Forest, `n_jobs` controls how many CPU cores they can use internally. Here, it's set to 1, meaning the model itself won't use multiple cores for its internal computations. The *script's* overall parallelism is handled by `joblib` later.
    - `solver='saga'`, `max_iter=1000` (for Logistic Regression), `verbose=0` (for CatBoost), `use_label_encoder=False`, `eval_metric='logloss'` (for XGBoost): These are specific settings for each model. For instance, `max_iter` defines the maximum number of iterations for the optimization algorithm. `verbose=0` means CatBoost won't print training progress, keeping the console clean.
  - **Second element**: A dictionary representing the *parameter grid* for `GridSearchCV`. These are the hyperparameters that `GridSearchCV` will systematically try out to find the best combination for that model. For example:
    - `"C": [0.01, 0.1, 1]` for Logistic Regression: `C` is the inverse of regularization strength. `GridSearchCV` will train Logistic Regression with `C=0.01`, then `C=0.1`, then `C=1`.
    - `"max_depth": [5, 10, 20]` for Decision Tree: `max_depth` limits how deep the tree can grow.

---

5. Feature Categories (`FEATURE_CATEGORIES`)

Python

```
# Feature categories (must match preprocessor)
FEATURE_CATEGORIES = {
    "flow_metrics": ["Flow Duration", "Flow Byts/s", "Flow Pkts/s", "Flow IAT
Mean", "Flow IAT Std", "Flow IAT Max", "Flow IAT Min"],
    # ... (other categories) ...
}
```

This dictionary defines different groups of features (columns) present in your datasets. This is a common practice in data science to perform experiments with different subsets of features to see their impact on model performance.

- **Key (e.g., `"flow_metrics"`)**: A descriptive name for the feature group.
- **Value (a list of strings)**: The actual names of the columns (features) that belong to that group. The comment "must match preprocessor" indicates that these feature names are expected to be present in the CSV files generated by a preceding data preprocessing step.

---

6. The Core Experiment Function (`run_experiment`)

Python
```
def run_experiment(params):
    file_path, group_name, feats, model_name, (estimator, grid) = params
    start_time = time.time()
    print(f"\n--- Starting {model_name} on {file_path.name}/{group_name} at
{time.ctime()} ---")

    # Load data
    df = pd.read_csv(file_path, low_memory=False)
    X = df[feats]
    y = df['Label']

    # Split
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, stratify=y, test_size=0.3, random_state=42
    )

    # Grid search
    print(f"Grid-searching {model_name} (params: {list(grid.keys())})...")
    gs = GridSearchCV(estimator, grid, cv=3, scoring='f1', n_jobs=1)
    gs.fit(X_train, y_train)
    best = gs.best_estimator_
    print(f"Best params for {model_name}: {gs.best_params_}")

    # Predict
    print(f"Evaluating {model_name}...")
    y_pred = best.predict(X_test)

    # Overall metrics
    overall = {
        'file': file_path.name,
        'attack_type': file_path.stem,
        'feature_group': group_name,
        'model': model_name,
        'best_params': json.dumps(gs.best_params_),
        'accuracy': accuracy_score(y_test, y_pred),
        'f1_score': f1_score(y_test, y_pred),
        'precision': precision_score(y_test, y_pred),
        'recall': recall_score(y_test, y_pred)
    }
```

```
    # Per-attack metrics
    per_attack = []
    df_test = df.loc[y_test.index].copy()
    df_test['_pred'] = y_pred
    for atk in sorted(df_test['Attack Type'].unique()):
        mask = df_test['Attack Type'] == atk
        if mask.sum() < 2: # Skip if less than 2 samples for this attack type
            continue
        pa_y = df_test.loc[mask, 'Label']
        pa_pred = df_test.loc[mask, '_pred']
        per_attack.append({
            'file': file_path.name,
            'attack_type': atk,
            'feature_group': group_name,
            'model': model_name,
            'accuracy': accuracy_score(pa_y, pa_pred),
            'f1_score': f1_score(pa_y, pa_pred),
            'precision': precision_score(pa_y, pa_pred),
            'recall': recall_score(pa_y, pa_pred)
        })

    elapsed = time.time() - start_time
    print(f"--- Completed {model_name} on {file_path.name}/{group_name} in
{elapsed:.1f}s ---")
    return overall, per_attack
```

This is the workhorse function. It encapsulates the entire training and evaluation process for a *single combination* of dataset, feature group, and model.

- **`def run_experiment(params):`**: Takes a single `params` tuple as input, which is then unpacked into individual variables. This design is crucial for `joblib.Parallel` later, as it expects a function that takes one argument.
- **`file_path, group_name, feats, model_name, (estimator, grid) = params`**: Unpacks the input tuple.
  - `file_path`: Path to the current dataset CSV.
  - `group_name`: Name of the feature group being used.
  - `feats`: List of actual feature names (columns) for the current group.
  - `model_name`: Name of the model (e.g., "LogisticRegression").
  - `estimator`: The scikit-learn model object (e.g., `LogisticRegression(...)`).
  - `grid`: The parameter grid for `GridSearchCV`.
- **`start_time = time.time()`**: Records the start time for performance measurement.
- **`df = pd.read_csv(file_path, low_memory=False)`**: Loads the CSV data into a pandas DataFrame. `low_memory=False` can prevent warnings with mixed data types in large files.
- **`X = df[feats]`**: Selects the feature columns (input to the model).
- **`y = df['Label']`**: Selects the target variable (what the model tries to predict). In this context, 'Label' likely indicates whether a network flow is benign or an attack.
- **`X_train, X_test, y_train, y_test = train_test_split(...)`**: Splits the data into training and testing sets.
  - `test_size=0.3`: 30% of the data will be used for testing, 70% for training.

    - `random_state=42`: Ensures the split is the same every time the script runs.
    - `stratify=y`: **Very important for classification tasks, especially with imbalanced classes.** This ensures that the proportion of different 'Label' values (e.g., attack vs. benign) is maintained in both the training and testing sets.
- **`gs = GridSearchCV(estimator, grid, cv=3, scoring='f1', n_jobs=1)`**: Initializes `GridSearchCV`.
    - `estimator`: The model to tune.
    - `grid`: The dictionary of hyperparameters to search.
    - `cv=3`: Uses 3-fold cross-validation. This means the training data is split into 3 parts, and the model is trained 3 times, each time using 2 parts for training and 1 for validation, averaging the results. This makes the hyperparameter selection more robust.
    - `scoring='f1'`: The metric used to judge the "best" parameters. Here, it's the F1-score, which is often preferred over accuracy for imbalanced classification problems (where one class is much more frequent than others).
    - `n_jobs=1`: `GridSearchCV` itself is told to use only one CPU core for its internal training loops *per fold*. The outer `joblib.Parallel` handles multiple *experiments* running in parallel.
- **`gs.fit(X_train, y_train)`**: Performs the actual grid search, training the model multiple times with different parameter combinations on the training data.
- **`best = gs.best_estimator_`**: Once the grid search is done, this attribute holds the trained model instance that performed best according to the `scoring` metric.
- **`print(f"Best params for {model_name}: {gs.best_params_}")`**: Reports the optimal hyperparameters found.
- **`y_pred = best.predict(X_test)`**: Uses the best-performing model to make predictions on the *unseen* test data.
- **Overall Metrics**: Calculates and stores overall performance metrics (accuracy, F1, precision, recall) for the entire test set.
    - `file`, `attack_type`, `feature_group`, `model`, `best_params`: Metadata for easy identification of the results. `json.dumps(gs.best_params_)` converts the dictionary of best parameters into a string for storage in a CSV.
- **Per-Attack Metrics**: This is a crucial part for an intrusion detection system. It goes beyond overall performance and calculates metrics for each *specific attack type* present in the test set.
    - `df_test = df.loc[y_test.index].copy()`: Creates a DataFrame of the test data, including the original 'Attack Type' column.
    - `df_test['_pred'] = y_pred`: Adds the model's predictions to this test DataFrame.
    - The loop iterates through unique attack types (`df_test['Attack Type'].unique()`).
    - For each attack type, it filters the test data (`mask`), extracts the true labels (`pa_y`) and predictions (`pa_pred`), and calculates metrics specifically for that attack type.
    - `if mask.sum() < 2: continue`: Skips calculation if there are too few samples for a given attack type, as metrics wouldn't be meaningful.
- **`return overall, per_attack`**: The function returns two dictionaries (or list of dicts), one for overall metrics and one for per-attack metrics.

7. Building the Task List

Python
```
# Build task list
tasks = []
for file_path in sorted(CLEAN_DIR.glob("*.csv")):
    print(f"Queueing file: {file_path.name}")
    for group, feats in FEATURE_CATEGORIES.items():
        sample_cols = pd.read_csv(file_path, nrows=1,
low_memory=False).columns
        avail = [f for f in feats if f in sample_cols]
        if not avail:
            print(f"  Skipping group {group}: no features present.")
            continue
        print(f"  Feature group: {group} ({len(avail)} features)")
        for model_name, model_def in MODELS.items():
            tasks.append((file_path, group, avail, model_name, model_def))

print(f"Total tasks to run: {len(tasks)}")
```

This section dynamically generates all possible combinations of datasets, feature groups, and models that need to be run. Each combination forms a "task" for the `run_experiment` function.

- **`for file_path in sorted(CLEAN_DIR.glob("*.csv")):`**: Iterates through all CSV files in the `CLEAN_DIR`. `glob("*.csv")` finds all files ending with `.csv`. `sorted()` ensures a consistent order.
- **`for group, feats in FEATURE_CATEGORIES.items():`**: For each file, it iterates through all defined feature categories.
- **`sample_cols = pd.read_csv(file_path, nrows=1, low_memory=False).columns`**: This is a clever optimization. Instead of loading the entire (potentially large) CSV file just to check its columns, it loads only the first row (`nrows=1`). This is much faster.
- **`avail = [f for f in feats if f in sample_cols]`**: It checks if the features defined in the `FEATURE_CATEGORIES` actually exist in the current CSV file. Datasets might not contain all possible features.
- **`if not avail: continue`**: If none of the features for a given group are present in the current file, that feature group is skipped for this file.
- **`for model_name, model_def in MODELS.items():`**: For each valid file and feature group, it iterates through all the defined machine learning models.
- **`tasks.append((file_path, group, avail, model_name, model_def))`**: Each unique combination (file, feature group, available features, model name, model definition) is added as a tuple to the `tasks` list. This list will then be fed to the parallel executor.

---

8. Running in Parallel: Speeding Things Up

Python
```
# Run in parallel
results = Parallel(n_jobs=3)(delayed(run_experiment)(t) for t in tasks)
```

This is where `joblib` comes into play to parallelize the execution.

- **`Parallel(n_jobs=3)`**: Initializes the parallel execution engine.
  - `n_jobs=3`: Tells `joblib` to use 3 CPU cores (or processes) to run tasks concurrently. If you have more cores, you could increase this number (e.g., `n_jobs=-1` to use all available cores).
- **`delayed(run_experiment)(t) for t in tasks`**: This is a generator expression that creates a list of "delayed" function calls. `delayed` is a wrapper that turns a function call into an object that `Parallel` can execute later.
- The entire line executes the `run_experiment` function for each task in the `tasks` list, but it does so in parallel across 3 processes. The `results` variable will store a list of the return values from each `run_experiment` call.

---

9. Separating and Saving Results

Python
```
# Separate results
overall_list, per_attack_list = zip(*results)

# Save DataFrames
print(f"Saving overall metrics to {OVERALL_CSV}")
pd.DataFrame(overall_list).to_csv(OVERALL_CSV, index=False)
print(f"Saving per-attack metrics to {PER_ATTACK_CSV}")
pd.DataFrame([row for sub in per_attack_list for row in
sub]).to_csv(PER_ATTACK_CSV, index=False)

print("All done!")
```

Finally, the script collects and saves the results.

- **`overall_list, per_attack_list = zip(*results)`**:
  - Recall that `run_experiment` returns a tuple: `(overall, per_attack)`.
  - `results` is a list of these tuples: `[(overall1, per_attack1), (overall2, per_attack2), ...]`.
  - `zip(*results)` "unzips" this list. It effectively transposes it, creating two separate tuples: one containing all the `overall` dictionaries, and another containing all the `per_attack` lists. These are then assigned to `overall_list` and `per_attack_list`.
- **`pd.DataFrame(overall_list).to_csv(OVERALL_CSV, index=False)`**:
  - `pd.DataFrame(overall_list)`: Converts the list of overall metric dictionaries into a single pandas DataFrame. Each dictionary becomes a row in the DataFrame.

    - `.to_csv(OVERALL_CSV, index=False)`: Saves this DataFrame to a CSV file. `index=False` prevents pandas from writing the DataFrame's index as a column in the CSV.
- **`pd.DataFrame([row for sub in per_attack_list for row in sub]).to_csv(PER_ATTACK_CSV, index=False)`**:
    - `per_attack_list` is a list of lists (each `per_attack` returned by `run_experiment` is itself a list of dictionaries).
    - `[row for sub in per_attack_list for row in sub]` is a list comprehension that *flattens* this list of lists into a single, continuous list of per-attack metric dictionaries.
    - This flattened list is then converted to a DataFrame and saved to `PER_ATTACK_CSV`.
- **`print("All done!")`**: A final confirmation message.

Summary for a Software Engineer:

From a software engineering perspective, this script demonstrates:

1. **Modular Design**: The `run_experiment` function encapsulates a complex process, making the code readable and maintainable.
2. **Configuration Management**: Clear separation of configurable parameters (directories, models, feature groups) from the core logic.
3. **Data Processing Pipelines**: Steps for data loading, splitting, training, prediction, and evaluation are clearly defined.
4. **Error Handling/Robustness**: Checking for available features, skipping sparse attack types, and using `random_state` for reproducibility contribute to robustness.
5. **Performance Optimization**: Leveraging `joblib` for parallel execution dramatically reduces the overall runtime by utilizing multiple CPU cores. This is akin to distributing a batch of jobs across worker threads/processes.
6. **Structured Output**: Saving results in well-structured CSV files makes it easy for downstream analysis and reporting.
7. **Iterative Experimentation**: The nested loops for files, feature groups, and models allow for a systematic exploration of different experimental conditions, a common pattern in scientific and data-driven projects.

In essence, this script is a well-engineered solution for systematically evaluating machine learning models on network intrusion detection data, providing comprehensive performance insights across various scenarios. It's a prime example of how software engineering principles are applied in the context of data science and machine learning projects.

Explain in detail joblib's Parallel and delayed. How is it used in this python script

As your tutor, let's dive deep into `joblib`'s `Parallel` and `delayed` functions, which are fantastic tools for speeding up Python code by utilizing multiple CPU cores. Think of it as a way to easily

distribute a set of independent tasks across your machine's processing power, much like how you might manage concurrent operations in a multi-threaded or multi-process application.

What are `joblib.Parallel` and `joblib.delayed`?

At their core, `Parallel` and `delayed` from the `joblib` library provide a simple and efficient way to parallelize loops in Python. They abstract away the complexities of managing processes or threads, inter-process communication, and result collection.

1. **`joblib.Parallel`**:
    - This is the main class that orchestrates the parallel execution.
    - You instantiate it, usually specifying the `n_jobs` parameter, which determines how many parallel jobs (processes or threads) should be used.
        - `n_jobs=1`: No parallelization; runs sequentially.
        - `n_jobs > 1`: Uses that many parallel jobs.
        - `n_jobs=-1`: Uses all available CPU cores. This is often the most efficient setting for CPU-bound tasks.
    - When called with a list of "delayed" functions, it distributes these function calls to the available jobs and collects their results.
2. **`joblib.delayed`**:
    - This is a simple helper function. It's a decorator or a function that wraps a regular function call.
    - When you do `delayed(my_function)(arg1, arg2)`, it doesn't immediately execute `my_function`. Instead, it creates a "delayed call" object. This object essentially holds the function reference and its arguments, waiting to be executed by a `Parallel` instance.
    - It acts as a placeholder that tells `Parallel` what function to call and with what arguments when it's its turn to run on one of the parallel processes.

Analogy:

Imagine you have a stack of reports (tasks) to process, and each report can be handled independently.

- **`joblib.Parallel`** is like your team of assistants. When you create `Parallel(n_jobs=3)`, you're saying, "I have 3 assistants ready to work."
- **`joblib.delayed`** is like putting a sticky note on each report. The sticky note says, "Take this report (`t`), and apply the `run_experiment` procedure to it."
- When you give the `Parallel` instance the stack of sticky-noted reports, your assistants (`n_jobs=3`) grab reports from the stack, process them according to their sticky notes (`run_experiment`), and then hand back the finished reports (results). All 3 assistants can work at the same time.

How is it used in *this* Python script?

Let's look at the relevant lines in the script:

Python

```
from joblib import Parallel, delayed
# ...
# Build task list
tasks = []
for file_path in sorted(CLEAN_DIR.glob("*.csv")):
    # ... (code to identify features and models) ...
    for model_name, model_def in MODELS.items():
        tasks.append((file_path, group, avail, model_name, model_def))

print(f"Total tasks to run: {len(tasks)}")

# Run in parallel
results = Parallel(n_jobs=3)(delayed(run_experiment)(t) for t in tasks)
# ... (rest of the script) ...
```

Here's the detailed breakdown of its usage:

1. **Preparation of Independent Tasks**:
    - The script first constructs a list called `tasks`.
    - Each element `t` in `tasks` is a tuple: `(file_path, group, avail, model_name, model_def)`.
    - Crucially, each of these tuples contains *all the necessary information* for a single run of the `run_experiment` function. This means that each `run_experiment` call is *independent* of the others. It doesn't modify shared state that other calls rely on, nor does it wait for results from other calls to proceed. This independence is key to effective parallelization.
    - The `tasks` list essentially defines the "workload" – all the unique combinations of data, features, and models that need to be processed.
2. **The Parallel Execution Line**:
    - `results = Parallel(n_jobs=3)(delayed(run_experiment)(t) for t in tasks)`

    Let's dissect this from right to left:

    - **`for t in tasks`**: This is a standard Python loop that iterates through each task tuple in the `tasks` list.
    - **`delayed(run_experiment)(t)`**: For each `t` (which is a tuple of parameters), this expression creates a `Delayed` object. It wraps the call to `run_experiment` with these specific parameters (`t`). It *does not* execute `run_experiment` immediately. Instead, it generates a sequence of "recipes" for future function calls.
        - Example: If `t1` is `(file1, groupA, featsX, LR, LR_params)` and `t2` is `(file1, groupA, featsX, DT, DT_params)`, this part generates `delayed(run_experiment)(t1)`, `delayed(run_experiment)(t2)`, and so on.
    - **`(...)` (Generator Expression)**: The entire `(delayed(run_experiment)(t) for t in tasks)` part is a generator expression. It doesn't create a full list of `Delayed` objects in memory all at once, but rather yields them one by one as `Parallel` requests them. This is memory efficient for a large number of tasks.

- **`Parallel(n_jobs=3)`**: This creates an instance of the `Parallel` class.
  - `n_jobs=3` means that `joblib` will spawn or use 3 separate processes (or threads, depending on the backend, but typically processes for CPU-bound tasks like ML training in Python to bypass GIL limitations).
  - These 3 processes will then pick up the `Delayed` calls from the generator one at a time.
- **`Parallel(...)(...)`**: When the `Parallel` instance is "called" with the sequence of `Delayed` objects, it starts its engine.
  - It distributes the `Delayed` calls among its `n_jobs` workers.
  - Each worker executes `run_experiment(t)` independently.
  - As each `run_experiment` call finishes, its return value (a tuple of `(overall, per_attack)` dictionaries) is collected by `Parallel`.
  - Once all tasks are completed by all workers, `Parallel` aggregates all the collected return values into a single list, which is then assigned to the `results` variable.

Why use `Parallel` and `delayed` here?

1. **CPU-Bound Tasks**: Training machine learning models is typically a CPU-intensive operation. Python's Global Interpreter Lock (GIL) limits true multi-threading for CPU-bound tasks in a single process. `joblib` overcomes this by using multiple *processes* instead of threads, where each process has its own Python interpreter and GIL, allowing true parallel execution on multi-core CPUs.
2. **Independent Tasks**: Each `run_experiment` call on a specific dataset/feature group/model combination is completely independent. It reads its own data, trains its own model, and calculates its own metrics without interfering with or needing data from other `run_experiment` calls. This makes it an ideal candidate for parallelization.
3. **Simplicity**: `joblib` provides a very clean and Pythonic API for parallelizing loops. You don't have to manually manage `multiprocessing.Pool`s, queues, or inter-process communication. You define your task function, wrap its calls with `delayed`, and pass them to `Parallel`.
4. **Efficiency**: By running 3 tasks concurrently (or however many cores you have), the total time to complete all experiments is significantly reduced compared to running them sequentially. If each experiment takes 10 minutes, and you have 3 cores, 3 experiments can finish in roughly 10 minutes instead of 30.

In summary, `joblib.Parallel` and `joblib.delayed` are used in this script to efficiently run a large number of independent machine learning model training and evaluation experiments concurrently across multiple CPU cores, dramatically speeding up the overall execution time of this comprehensive benchmarking process.

If you are using Ubuntu 25.04 with AMD Ryzen 5 3500X CPU and Radeon RX 6400 GPU system to run this python code, In what ways we can improve this code to maximize the resource usage. Provide the complete updated code

Let's break down how to optimize this Python code for your specific Ubuntu 25.04 system with an AMD Ryzen 5 3500X CPU and Radeon RX 6400 GPU, maximizing resource utilization.

Understanding Your System's Resources:

- **AMD Ryzen 5 3500X CPU:** This is a 6-core, 6-thread CPU. This means it has 6 physical cores, and it can handle 6 simultaneous threads effectively (no SMT/Hyperthreading on this specific model like some other Ryzens). This is crucial for `joblib.Parallel`'s `n_jobs`.
- **Radeon RX 6400 GPU:** This is a dedicated graphics card. Python's `scikit-learn` (used for most models here) is primarily CPU-bound. However, some libraries like XGBoost and LightGBM *can* leverage GPUs for training, which can provide significant speedups for larger datasets and more complex models. Ubuntu 25.04 should have good driver support for modern AMD GPUs.

Optimization Strategies and Changes:

Here's how we'll modify the code, focusing on maximizing both CPU and (where applicable) GPU usage:

1. **Maximize CPU Cores for `joblib.Parallel`:**
   - The Ryzen 5 3500X has 6 physical cores. Setting `n_jobs=-1` for `joblib.Parallel` will tell it to use all available CPU cores, which is 6 in your case. This is the primary way to parallelize the *entire set of experiments*.
2. **Enable GPU Acceleration for XGBoost, LightGBM, and CatBoost:**
   - These three gradient boosting libraries support GPU training. To utilize your Radeon RX 6400, we'll need to configure them correctly.
     - **XGBoost:** Set `tree_method='gpu_hist'` and `gpu_id=0` (assuming one GPU).
     - **LightGBM:** Set `device='gpu'` and `gpu_platform_id=0`, `gpu_device_id=0`.
     - **CatBoost:** Set `task_type='GPU'`.
   - **Important Prerequisites for GPU:**
     - **ROCm (Radeon Open Compute) Drivers:** For AMD GPUs, you need AMD's ROCm platform installed and correctly configured on Ubuntu. This is the equivalent of NVIDIA's CUDA. Ubuntu 25.04 should make this process relatively straightforward. Without ROCm, these `gpu` settings will either fall back to CPU or raise errors.
     - **Library Versions:** Ensure you have `xgboost`, `lightgbm`, and `catboost` versions installed that are compiled with ROCm/GPU support. You might need to install specific `pip` packages or build from source if the default `pip install` doesn't include GPU binaries for ROCm. For example: `pip install xgboost lightgbm catboost` will usually give you CPU versions by default. You might need to check their respective documentation for GPU-enabled installation instructions (e.g., `pip`

install xgboost[amdgpu] or similar, depending on future packaging). For now, we'll assume the basic settings work if ROCm is present.

3. **Adjust `n_jobs` for Individual Classifiers (if applicable):**
   - Some `scikit-learn` models (like `RandomForestClassifier`, `ExtraTreesClassifier`, `KNeighborsClassifier`) have an `n_jobs` parameter *within their constructors*.
   - **Crucial Point:** When using `joblib.Parallel` *externally* to run multiple *independent* experiments, you generally want the `n_jobs` of the *internal* models to be `1`. This is because `joblib.Parallel` is already distributing the work across processes. If an internal model also tries to use multiple cores (`n_jobs=-1`), you'll end up with "over-subscription" – more threads/processes trying to run than you have physical cores, leading to context switching overhead and potentially *slower* overall execution.
   - In the provided code, `RandomForestClassifier` and `ExtraTreesClassifier` already have `n_jobs=1`, which is good. `KNeighborsClassifier` doesn't have `n_jobs` in its constructor, but its `fit` and `predict` methods can sometimes benefit from `n_jobs` if passed, though it's less common for basic usage. `MLPClassifier` is usually single-threaded on the CPU unless using a specialized backend.
   - For `GridSearchCV`, its `n_jobs` parameter refers to how many parallel jobs *GridSearchCV itself* will use *for each individual model's hyperparameter search*. Setting `n_jobs=-1` here would mean that within *one* `run_experiment` call, `GridSearchCV` tries to use all cores. Given that `run_experiment` calls are already parallelized by the outer `joblib.Parallel`, keeping `GridSearchCV`'s `n_jobs=1` is typically the best strategy to avoid resource contention and over-subscription. We want `joblib` to parallelize the *experiments*, not for each experiment to parallelize its *internal* grid search excessively.
4. **Logging and Verbosity:**
   - For `CatBoostClassifier`, `verbose=0` is already good for reducing console spam.
   - For XGBoost, `eval_metric='logloss'` and `use_label_encoder=False` are standard practice. We will keep these.

Updated Code:

Python
```
# run_all_base_models.py

# Robust driver to train and evaluate multiple classifiers on all cleaned
CIC-IDS-2018 datasets
# across defined feature groups. Computes overall and per-attack metrics and
saves results.

import pandas as pd
from pathlib import Path
from joblib import Parallel, delayed # For CPU-based parallelization of
experiments
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, ExtraTreesClassifier
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from catboost import CatBoostClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import LinearSVC
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, f1_score, precision_score,
recall_score, confusion_matrix
import json
import time
import os # To potentially check CPU core count

# --- Configuration ---
CLEAN_DIR = Path("cleaned")
RESULTS_DIR = Path("results") / "base_models"
RESULTS_DIR.mkdir(parents=True, exist_ok=True)
OVERALL_CSV = RESULTS_DIR / "overall_metrics.csv"
PER_ATTACK_CSV = RESULTS_DIR / "per_attack_metrics.csv"

# Determine optimal n_jobs for overall Parallel execution
# For a Ryzen 5 3500X (6 Cores, 6 Threads), n_jobs=-1 will use 6 processes.
# This should be the primary level of parallelism.
# We will generally keep internal model n_jobs=1 to avoid oversubscription.
NUM_CPU_CORES = os.cpu_count() if os.cpu_count() is not None else 1
# Use all detected CPU cores for joblib.Parallel
JOBLIB_N_JOBS = NUM_CPU_CORES # -1 means use all available cores, equivalent
to NUM_CPU_CORES for CPU-bound.
                              # Explicitly using NUM_CPU_CORES can sometimes
be clearer.
print(f"Detected {NUM_CPU_CORES} CPU cores. Using {JOBLIB_N_JOBS} for
joblib.Parallel.")


# Classifiers and parameter grids
MODELS = {
    "LogisticRegression": (LogisticRegression(solver='saga', max_iter=1000,
random_state=42), {"C": [0.01, 0.1, 1]}),
    "DecisionTree": (DecisionTreeClassifier(random_state=42), {"max_depth":
[5, 10, 20]}),
    # RandomForest and ExtraTrees already have n_jobs=1, which is correct for
internal models
    "RandomForest": (RandomForestClassifier(random_state=42, n_jobs=1),
{"n_estimators": [100, 200], "max_depth": [None, 10, 20]}),
    "ExtraTrees": (ExtraTreesClassifier(random_state=42, n_jobs=1),
{"n_estimators": [100, 200], "max_depth": [None, 10, 20]}),

    # Enable GPU for XGBoost
    # Requires ROCm (AMD's CUDA equivalent) installed and GPU-enabled XGBoost
package
    "XGBoost": (XGBClassifier(use_label_encoder=False, eval_metric='logloss',
random_state=42, n_jobs=1,
                              tree_method='gpu_hist', gpu_id=0), # Added for
GPU
              {"learning_rate": [0.1, 0.01], "n_estimators": [100, 200],
"max_depth": [3, 5]}),
```

```
    # Enable GPU for LightGBM
    # Requires ROCm and GPU-enabled LightGBM package
    "LightGBM": (LGBMClassifier(random_state=42, n_jobs=1,
                                device='gpu', gpu_platform_id=0,
gpu_device_id=0), # Added for GPU
                {"learning_rate": [0.1, 0.01], "n_estimators": [100, 200],
"num_leaves": [31, 50]}),

    # Enable GPU for CatBoost
    # Requires ROCm and GPU-enabled CatBoost package
    "CatBoost": (CatBoostClassifier(verbose=0, random_state=42,
                                    task_type='GPU'), # Added for GPU
                {"depth": [4, 6], "iterations": [100, 200], "learning_rate":
[0.1, 0.01]}),

    "KNN": (KNeighborsClassifier(), {"n_neighbors": [3, 5, 7]}),
    "LinearSVC": (LinearSVC(max_iter=10000, random_state=42), {"C": [0.1, 1,
10]}),
    "MLP": (MLPClassifier(max_iter=500, random_state=42),
{"hidden_layer_sizes": [(50,), (100,)], "alpha": [1e-4, 1e-3]})
}

# Feature categories (must match preprocessor)
FEATURE_CATEGORIES = {
    "flow_metrics": ["Flow Duration", "Flow Byts/s", "Flow Pkts/s", "Flow IAT
Mean", "Flow IAT Std", "Flow IAT Max", "Flow IAT Min"],
    "packet_size_stats": ["Tot Fwd Pkts", "Tot Bwd Pkts", "TotLen Fwd Pkts",
"TotLen Bwd Pkts", "Fwd Pkt Len Max", "Fwd Pkt Len Min", "Fwd Pkt Len Mean",
"Fwd Pkt Len Std", "Bwd Pkt Len Max", "Bwd Pkt Len Min", "Bwd Pkt Len Mean",
"Bwd Pkt Len Std", "Pkt Len Min", "Pkt Len Max", "Pkt Len Mean", "Pkt Len
Std", "Pkt Len Var", "Pkt Size Avg", "Fwd Seg Size Avg", "Bwd Seg Size Avg"],
    "timing_iat": ["Fwd IAT Tot", "Fwd IAT Mean", "Fwd IAT Std", "Fwd IAT
Max", "Fwd IAT Min", "Bwd IAT Tot", "Bwd IAT Mean", "Bwd IAT Std", "Bwd IAT
Max", "Bwd IAT Min"],
    "flags_and_protocol": ["FIN Flag Cnt", "SYN Flag Cnt", "RST Flag Cnt",
"PSH Flag Cnt", "ACK Flag Cnt", "URG Flag Cnt", "CWE Flag Count", "ECE Flag
Cnt", "Fwd PSH Flags", "Bwd PSH Flags", "Fwd URG Flags", "Bwd URG Flags",
"Protocol", "Dst Port", "Init Fwd Win Byts", "Init Bwd Win Byts"],
    "rates_and_ratios": ["Down/Up Ratio", "Fwd Pkts/s", "Bwd Pkts/s", "Fwd
Byts/b Avg", "Fwd Pkts/b Avg", "Fwd Blk Rate Avg", "Bwd Byts/b Avg", "Bwd
Pkts/b Avg", "Bwd Blk Rate Avg"],
    "connection_activity": ["Subflow Fwd Pkts", "Subflow Fwd Byts", "Subflow
Bwd Pkts", "Subflow Bwd Byts", "Fwd Act Data Pkts", "Active Mean", "Active
Std", "Active Max", "Active Min", "Idle Mean", "Idle Std", "Idle Max", "Idle
Min"]
}

# Function to train and evaluate one combination
def run_experiment(params):
    file_path, group_name, feats, model_name, (estimator, grid) = params
    start_time = time.time()
    print(f"\n--- Starting {model_name} on {file_path.name}/{group_name} at
{time.ctime()} ---")

    # Load data
    df = pd.read_csv(file_path, low_memory=False)
```

```
    X = df[feats]
    y = df['Label']

    # Split
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, stratify=y, test_size=0.3, random_state=42
    )

    # Grid search
    print(f"Grid-searching {model_name} (params: {list(grid.keys())})...")
    # For GridSearchCV, keep n_jobs=1. The outer joblib.Parallel handles
multiple experiments.
    # Setting GridSearchCV's n_jobs to -1 would cause oversubscription of CPU
cores.
    gs = GridSearchCV(estimator, grid, cv=3, scoring='f1', n_jobs=1)
    gs.fit(X_train, y_train)
    best = gs.best_estimator_
    print(f"Best params for {model_name}: {gs.best_params_}")

    # Predict
    print(f"Evaluating {model_name}...")
    y_pred = best.predict(X_test)

    # Overall metrics
    overall = {
        'file': file_path.name,
        'attack_type': file_path.stem,
        'feature_group': group_name,
        'model': model_name,
        'best_params': json.dumps(gs.best_params_),
        'accuracy': accuracy_score(y_test, y_pred),
        'f1_score': f1_score(y_test, y_pred),
        'precision': precision_score(y_test, y_pred),
        'recall': recall_score(y_test, y_pred)
    }

    # Per-attack metrics
    per_attack = []
    df_test = df.loc[y_test.index].copy()
    df_test['_pred'] = y_pred
    # Ensure 'Attack Type' column exists for per-attack metrics
    if 'Attack Type' in df_test.columns:
        for atk in sorted(df_test['Attack Type'].unique()):
            mask = df_test['Attack Type'] == atk
            # Ensure there are at least 2 samples of the attack type for
meaningful metric calculation
            if mask.sum() < 2:
                continue
            pa_y = df_test.loc[mask, 'Label']
            pa_pred = df_test.loc[mask, '_pred']
            # Only calculate metrics if both true and predicted labels have
variance
            # (i.e., not all labels are the same, which can happen with very
few samples)
            if len(set(pa_y)) > 1 or len(set(pa_pred)) > 1:
                per_attack.append({
                    'file': file_path.name,
```

```
                    'attack_type': atk,
                    'feature_group': group_name,
                    'model': model_name,
                    'accuracy': accuracy_score(pa_y, pa_pred),
                    'f1_score': f1_score(pa_y, pa_pred),
                    'precision': precision_score(pa_y, pa_pred),
                    'recall': recall_score(pa_y, pa_pred)
                })
            else:
                 print(f"  Skipping per-attack metrics for '{atk}' (file:
{file_path.name}) due to insufficient label variance.")
    else:
        print(f"  Warning: 'Attack Type' column not found in
{file_path.name}. Skipping per-attack metrics for this file.")


    elapsed = time.time() - start_time
    print(f"--- Completed {model_name} on {file_path.name}/{group_name} in
{elapsed:.1f}s ---")
    return overall, per_attack

# Build task list
tasks = []
for file_path in sorted(CLEAN_DIR.glob("*.csv")):
    print(f"Queueing file: {file_path.name}")
    for group, feats in FEATURE_CATEGORIES.items():
        try:
            # Load only a small sample to get columns, faster than loading
entire file
            sample_df = pd.read_csv(file_path, nrows=1, low_memory=False)
            sample_cols = sample_df.columns
        except Exception as e:
            print(f"  Error reading columns from {file_path.name}: {e}.
Skipping this file.")
            continue # Skip to next file if unable to read columns

        avail = [f for f in feats if f in sample_cols]
        # Also ensure 'Label' column exists
        if 'Label' not in sample_cols:
            print(f"  Skipping file {file_path.name}: 'Label' column not
found.")
            continue
        # And 'Attack Type' for per-attack metrics later
        if 'Attack Type' not in sample_cols:
             print(f"  Warning: 'Attack Type' column not found in
{file_path.name}. Per-attack metrics might be incomplete for this file.")

        if not avail:
            print(f"  Skipping group {group} for {file_path.name}: no
features present.")
            continue
        print(f"  Feature group: {group} ({len(avail)} features)")
        for model_name, model_def in MODELS.items():
            tasks.append((file_path, group, avail, model_name, model_def))

print(f"Total tasks to run: {len(tasks)}")
```

```
# Run in parallel, using all detected CPU cores
results = Parallel(n_jobs=JOBLIB_N_JOBS)(delayed(run_experiment)(t) for t in
tasks)

# Separate results
overall_list, per_attack_list_nested = zip(*results) # Renamed to avoid
confusion with flattening

# Filter out None results if any experiment failed or returned None
overall_list = [o for o in overall_list if o is not None]
per_attack_list_flat = [row for sublist in per_attack_list_nested for row in
sublist if row is not None]


# Save DataFrames
if overall_list:
    print(f"Saving overall metrics to {OVERALL_CSV}")
    pd.DataFrame(overall_list).to_csv(OVERALL_CSV, index=False)
else:
    print("No overall metrics to save.")

if per_attack_list_flat:
    print(f"Saving per-attack metrics to {PER_ATTACK_CSV}")
    pd.DataFrame(per_attack_list_flat).to_csv(PER_ATTACK_CSV, index=False)
else:
    print("No per-attack metrics to save.")

print("All done!")
```

Explanation of Changes and Why They Maximize Resources:

1. **`import os` and `NUM_CPU_CORES = os.cpu_count()`:**
    - We now dynamically detect the number of CPU cores using `os.cpu_count()`. This makes the script more portable.
    - `JOBLIB_N_JOBS = NUM_CPU_CORES`: This ensures that `joblib.Parallel` will spin up as many worker processes as you have physical cores on your Ryzen 5 3500X (which is 6). Each worker process will handle one `run_experiment` task independently. This is the primary knob for maximizing CPU utilization.
2. **GPU Configuration in `MODELS` Dictionary:**
    - **XGBoost:**

      Python

      ```
      "XGBoost": (XGBClassifier(..., tree_method='gpu_hist', gpu_id=0),
      ...)
      ```

      `tree_method='gpu_hist'` tells XGBoost to use the GPU-accelerated histogram-based algorithm, which is highly efficient. `gpu_id=0` specifies which GPU to use (0 for the first GPU found).

    - **LightGBM:**

Python

```
"LightGBM": (LGBMClassifier(..., device='gpu', gpu_platform_id=0, gpu_device_id=0), ...)
```

`device='gpu'` instructs LightGBM to use the GPU. `gpu_platform_id` and `gpu_device_id` are sometimes needed to select the correct OpenCL platform/device, especially in multi-GPU setups or if you have integrated graphics alongside a dedicated card. For a single dedicated RX 6400, `0` for both is usually correct.

- **CatBoost:**

  Python

  ```
  "CatBoost": (CatBoostClassifier(..., task_type='GPU'), ...)
  ```

  `task_type='GPU'` is the straightforward way to enable GPU training in CatBoost.

3. **`GridSearchCV` `n_jobs` (Unchanged, but important rationale):**
   - The `n_jobs=1` inside `GridSearchCV` remains crucial. If you set `GridSearchCV(..., n_jobs=-1)` *and* `Parallel(n_jobs=NUM_CPU_CORES)`:
     - You'd have `NUM_CPU_CORES` `run_experiment` processes running concurrently.
     - *Each* of those `NUM_CPU_CORES` processes would then try to use `NUM_CPU_CORES` cores for its internal `GridSearchCV`.
     - This leads to `NUM_CPU_CORES * NUM_CPU_CORES` total CPU processes/threads being requested, which is severe oversubscription (e.g., 6×6=36 processes on a 6-core CPU). This causes massive context switching overhead and will significantly *slow down* your total execution time.
   - By keeping `GridSearchCV`'s `n_jobs=1`, we ensure that `joblib.Parallel` handles the coarse-grained parallelism (running multiple independent experiments simultaneously), while each individual experiment performs its grid search sequentially (or relies on the GPU for the accelerated models). This is the optimal strategy.
4. **Robustness Improvements (Minor but Good Practice):**
   - Added `try-except` block for `pd.read_csv(nrows=1)` to gracefully handle unreadable CSV files.
   - Added checks for `'Label'` and `'Attack Type'` columns. It's good to inform the user if critical columns are missing.
   - Improved the per-attack metrics to skip calculation if there's insufficient label variance (e.g., only one unique label in `pa_y` or `pa_pred` for a given attack type), which can lead to errors in metric functions.
   - Filtered `None` results from `overall_list` and `per_attack_list_nested` before creating DataFrames, in case any `run_experiment` call failed or returned `None`.

- Added checks before saving DataFrames to prevent creating empty CSVs if no results were collected.

To run this optimized code and truly maximize GPU usage, ensure you have:

1. **ROCm (Radeon Open Compute) Installed on Ubuntu 25.04:** Follow AMD's official documentation for installing ROCm on your specific Ubuntu version. This is non-trivial but essential for GPU acceleration.
2. **GPU-enabled Python Packages:** After installing ROCm, you might need to install `xgboost`, `lightgbm`, and `catboost` in a way that links them against your ROCm installation. This often involves specific `pip` commands or building from source, as the default `pip install` usually provides CPU-only versions. Check each library's documentation for "GPU" or "ROCm" installation instructions.
   - For example, `pip install xgboost lightgbm catboost` might work if the default wheels now include ROCm support (increasingly common). If not, you might look for `pip install xgboost[amdgpu]` or similar, or refer to their build instructions.

By implementing these changes and ensuring the underlying ROCm and GPU-enabled library installations are correct, you will be effectively leveraging both your AMD Ryzen CPU's 6 cores and your Radeon RX 6400 GPU for the most demanding parts of the machine learning training, leading to significantly faster execution of your experiments.